

Vision-Based Drone Landing on a Moving Platform

MTP detailed report (V5): problem statement, approach, everything done and tried, findings, current work and file map

Yashwanth · status as of 6 October 2026 · repo:
github.com/yashwanth73337/drone-landing-rl, folder v5_shin/

About this document. A detailed, self-contained briefing for the supervisor or a second reviewer. It states the problem precisely, explains the approach and its rules, lists every file built and what it does, and reports every result with the checkpoint, timestep and task it was measured on. Negative and ambiguous results are reported as they are. Results are single-seed unless stated otherwise.

Short version.

- **Goal:** a drone lands on a moving platform using only its own camera and onboard sensors.
Supervisor's question: when this fails, is it a **state** (estimation) problem or a **control** problem?
- **Approach:** V5, a faithful from-scratch reimplementation of Shin et al. (IEEE RA-L 2026), in simulation. The same learning setup is trained with the **true** relative state (variant P, the control ceiling) and with **vision** (variant A), then compared on identical pinned test episodes.
- **Built:** all 14 blocks: simulator, sensors, reward, curriculum, network, recurrent PPO with the estimation loss, active-perception reward, evaluation and diagnostics. **175 automated tests pass.**
- **Main result (3 seeds each):** with perfect information the policy lands 97–100% up to $c = 0.625$ and 85% at full difficulty. Vision (A) lands 92% / 83% / 74% / 45% at $c = 0.125$ / 0.5 / 0.625 / 1.0: a gap of 7.5 → 40 points that grows with platform motion. Vision reached only level 50 / 50 / 30 of 80 in 4M steps.
- **Without the estimation loss, vision does not learn at all:** A-noEst stayed at level 10 for 4M steps and lands 25% even there (4.5% at full difficulty). Learning to extract the platform's state from images is the essential, hard part.
- **State or control?** Two-part answer so far. (1) During *learning*, state is the bottleneck: without explicit state supervision the vision policy never learns. (2) Once trained with it, correcting A's explicit estimate at test time recovers only ~13% of its remaining gap to the true-state policy (3-seed mean, never more than 27%), so that residual gap lies in what the vision-trained network does with its representation, not in the 6-number estimate.
- **Second finding:** every policy lands too hard (2–4 m/s; ≤ 1 m/s in under 2.5% of episodes). The cause: the paper's reward pays for descent and its success rule has no speed limit, and the policies exploit an asymmetry in the simulated motors (AerialGym's own LMF2 model) to fall faster than they can command. The paper's success metric hides this.
- **Just finished (6 Oct):** vision seeds 2 and 3 with the true-state test; the conclusions now rest on three seeds. **Next:** a curriculum-matched comparison (P checkpoints saved at the same level A reached), then a choice of the next long run.

1. Problem statement

1.1 The task

A quadrotor starts 2–8 m above and up to 3 m to the side of a ground vehicle carrying a 1.5×1.5 m landing pad. The vehicle drives with a randomly varying speed (up to 8 m/s) and turning rate, which are not known to the drone. The drone must touch down on the pad's top surface within 30 s, using only:

- a forward-facing grayscale camera pitched 60° down (512×320 pixels, 90° horizontal field of view), and
- its own body-frame velocity and attitude (noisy, as from an onboard estimator).

No GPS, no motion capture, no communication with the vehicle. The pad is seen only when it is in the camera's field of view; near touchdown and during turns it is often out of view (measured: about half of the frames of a successful landing).

1.2 Formal setting (as in the reference paper)

Element	Definition
Decision process	Partially observable MDP, decision step $\Delta t = 0.1$ s (10 Hz), horizon 300 steps (30 s).
Observation (actor)	$o_t = (I_t, u_t)$: image I_t ; $u_t = [\text{body velocity (3), attitude quaternion (4)}]$ with noise 0.05 m/s and 0.5° .
Hidden state that matters	$s_{\text{rel}} = [\Delta x, \Delta v]$: platform position and velocity relative to the drone, in the drone's body frame (6 numbers).
Action	$a_t = [v_x, v_y, v_z, \text{yaw rate}]$ velocity commands in the heading frame, executed by a geometric (Lee) controller.
Success	Paper: any contact with the pad's upper surface. We also report strict success (centre of mass over the pad) and touchdown speed.
Failure	Crash into ground or platform side, tilt $> 80^\circ$, drifting away (> 15 m), or timeout.
Difficulty	Curriculum scalar $c \in [0, 1]$ scales platform speed and its random changes; $c = 1$ is the full task.

1.3 The research question

When vision-based landing on a moving platform fails, is it a **STATE problem**: the drone does not know accurately enough where the platform is and how it moves (perception/estimation)? Or is it a **CONTROL problem**: even with a good estimate, the learned policy, or the way it is trained, does not produce a landing?

Why it matters: the two answers lead to different fixes (better perception and estimation, vs a better controller, reward or training). Earlier work (V1–V4, Section 2) could not separate them.

Thesis claim built alongside: *binary success rate is a misleading metric for this task*. Policies with equal success can fail very differently, and a "success" can be an unsafe impact.

2. Background: why V5

Stage	What changed	Outcome
Strand B	Lander.AI rebuild (TD3, true state, Crazyflie)	100% stationary, 98.7% at 0.25 m/s, 3 seeds. Frozen reference.
V1	Camera attached, observes only	Pad visible only ~half of a landing.
V2	Frozen policies on ArUco + dead reckoning	Vision costs ~45 points; crashes onto stale pad positions.
V3a	TD3 retrained on the vision observation	Collapsed into hovering.
V3b	Offline-trained LSTM estimator	85% stationary; on a moving pad crashes return; later collapsed to hovering even with the true state.
V4	Shin et al. method with many deviations	Estimator worked; policy never landed.

V4 deviated from the paper in action type, camera, drone, platform motion and more, so its failure could not be attributed. **V5 rebuilds the paper faithfully from scratch, with no V1-V4 code.**

3. Approach

3.1 Reference method (Shin et al., RA-L 11(5), 2026)

Part	Paper (reproduced in V5 unless noted)
Simulator	AerialGym (Isaac Gym). V5: PyBullet with AerialGym's LMF2 drone and controller copied over (deviation: no Isaac Gym on the lab desktop).
Network	CNN → LSTM (512) → estimation layer whose first 6 outputs estimate s_{rel} → decision layer → action. Critic sees the true state (training only).
Learning	Recurrent PPO + auxiliary estimation loss L_{est} (MSE between estimate and true s_{rel}).
Active perception	Extra reward $-0.1 \cdot \text{clip}(L_{est}(t+1) - 0.01, 0, 1)$: penalises actions that make the next estimate worse.
Reward	+10 landing, -10 crash/drift; shaping: lateral progress, vertical progress, vertical-speed penalty, undershoot, yaw rate.
Curriculum	Levels 10...80 ($c = \text{level}/80$), promote at $\geq 80\%$ success over 512 episodes.
Domain randomisation	Controller gains, wind-like forces/torques, initial velocities, ground textures, lighting, colour.
Landing pad	PACMAN keypoint pad with a frozen keypoint encoder. V5 base: ArUco marker + CNN = the paper's own "w/o keypoint encoder" ablation (PACMAN weights are not public).
Paper results	Full method 97%; ArUco + CNN 91%; without state estimation 73.99%; EKF + RL 59.77%.

3.2 How V5 answers the question

Variant	What the actor sees	Role
P (privileged)	True $s_{rel} + u_t$; no camera	Control ceiling: can this reward/PPO/curriculum learn to land at all?
A (proposed, ArUco base)	Image + u_t ; learned estimator + active perception	The vision system to diagnose.
A + true-state injection	A's trained network, but its estimate replaced by the truth (evaluation only)	Does correcting the estimate close the gap? Yes → state; no → control.
A-noEst	A without the estimation loss (and so without active perception)	Does the learned estimator help at all? (paper: 73.99% vs 91%)
A-noAP, H (EKF + RL), K (PACMAN)	Further ablations / baselines	Planned.

3.3 Rules followed throughout

- **Faithful first.** Every place where the paper is silent, ambiguous or cannot be matched is a numbered decision (D1-D18), recorded with its reason.
- **Block by block, tested.** Nothing is built on an untested block.
- **One change at a time** between any two compared runs.
- **Pinned evaluation:** 1,000 episodes, episode i uses seed $9000 + i$, deterministic actions, so all variants face identical episodes. Every number names its checkpoint, timestep and task.
- **More than the paper's metric:** failure breakdown, strict success, touchdown speed and safe success, plus estimation error by how long the pad has been out of view.
- **No training run or parameter change without the owner's approval.**

4. Key decisions (D1-D18)

ID	Decision
D1	Own gymnasium environment on raw PyBullet (not gym-pybullet-drones).
D2	Drone: AerialGym LMF2 (1.24 kg); the paper's controller gains are exactly LMF2's.
D3 / D17	Vertical-speed penalty: use the equation as printed $(-[v_z + 0.5]^+)$, which pays for descent). The prose reading was tested (D17) and rejected: 30.5% vs 81.5% success.
D4	Δz = pad top – drone height (negative while above).
D5, D6	Curriculum levels 10...80, promote at $\geq 80\%$ over 512 episodes; platform speed ≤ 8 c m/s, turn rate $\leq 30^\circ/\text{s}$.
D7	Torque disturbance read as $\pm 4\text{e-}3$ N·m (typo in the paper's table).
D8-D10	Own recurrent PPO; render 512×320 and downsample to 256×160 ; estimation-loss weight 1.0.
D11	Include variant P (true-state actor) as the control ceiling.
D12, D13	Start facing the pad $\pm 15^\circ$ with the pad in view; 50 procedural ground textures.
D14	3 seeds for any reported configuration; 1 for exploratory runs.
D15	Success = paper's definition; strict success always reported (the 0.5 m drone body can touch the pad with its centre off the pad).
D16	Vertical speed command limited to ± 1 m/s (paper silent; its Fig. 9 shows ≤ 1 m/s descents).
D18	Vision training renders on the CPU (each GPU renderer costs ~ 250 MB; 16 of them do not fit on the 4 GB T400 alongside training). Same overall speed; no PPO change.

5. What we did, step by step

Chronological record of the V5 work. Dates are 2026. Each step was approved before any training run.

When	Step	Outcome
29 Sep	Migrated the project to the lab desktop (T400 4 GB GPU, 20 CPU threads); fixed a .gitignore rule that had silently untracked V3b's estimator code.	Desktop reproduces the laptop's V3b numbers.
30 Sep	Read the paper and wrote SPEC.md: every parameter tagged [paper] / [inferred] / [unspecified] / [deviation]; decisions D1-D15 taken.	Spec agreed before any code.
30 Sep	Built and tested Blocks 1-12 and 14: drone + controller, platform, camera + ArUco, termination, oracle, domain randomisation, CNN, environment, reward, curriculum, network, recurrent PPO, evaluation.	160 tests; oracle 99% at full difficulty.
30 Sep	First training run P_smoke_s1 (true-state actor, ± 3 m/s vertical limit).	91% success at $c = 1$, but dives: impact -4.1 m/s.

When	Step	Outcome
30 Sep	D16: vertical command limited to ± 1 m/s. Run P_smoke_vz1_s1 .	81.5%; impact still -2.3 m/s, faster than it can command.
1 Oct	D17 test: the prose reading of the vertical-speed penalty, run P_smoke_prose_vz1_s1 .	30.5%, stuck at level 10, many flips. Rejected; literal reading kept.
1 Oct	Status report and second opinion; audit of coordinate signs and of the motor model against AerialGym's source code.	Signs consistent; motor constants match AerialGym exactly.
1 Oct	Motor diagnostic (evaluation only): same checkpoint flown with symmetric motors.	Touchdown $-2.30 \rightarrow -0.78$ m/s: the motor asymmetry explains the excess speed.
1 Oct	Training with symmetric motors, run P_smoke_vz1_symslow_s1 .	Still -1.93 m/s: the root cause is the reward / success rule, not the motors.
1 Oct	Built Block 13 (active-perception reward) with a hand-checked test.	173 tests.
1 Oct	First vision launch crashed out of GPU memory; measured ~ 250 MB per GPU-rendered environment. D18: render on the CPU. Smoke run A_smoke_s1 (100k steps).	Vision pipeline works at ~ 57 steps/s (~ 4.9 h per million steps).
1-3 Oct	Main vision run A_s1 (4M steps, 20 h).	Reached level 50 of 80; 88.8% at $c = 0.5$, 48.5% at $c = 1$.
3 Oct	Same-episode comparison of P and A at four difficulties; built the true-state injection diagnostic; flew A on symmetric motors.	Injection recovers only 0-27% of A's gap; A also relies on the motor sink.
3-4 Oct	Queue: P_smoke_vz1_s2 , P_smoke_vz1_s3 , then A_noEst_s1 (4M steps).	P stable (85.1% mean at $c = 1$); A-noEst stuck at level 10, 4.5% at $c = 1$.
4 Oct	Status reports written; results recorded in SPEC; report committed under v5_shin/reports/.	-
4-6 Oct	Queue 2: vision runs A_s2 and A_s3 with evaluations and the injection test.	Levels 50 and 30; injection recovers $\sim 13\%$ of the gap on average (Section 8.5).

6. Problems found along the way and how they were fixed

Each of these would have silently corrupted results; each now has a regression test or a recorded check.

Problem	Effect if missed	Fix
PyBullet computed the drone's inertia from its collision box	Inertia $4\times$ too large; wrong attitude dynamics	Load with URDF_USE_INERTIA_FROM_FILE; test checks inertia
GPU renderer drew blank frames; textures shifted; marker sheared; surfaces flickered (z-fighting)	Images without a usable marker	Load renderer first; no URDF textures; 768 px RGB texture; single pad mesh; hidden ground collision visual
Touchdown speed read after the contact solver zeroed it	Impact speed reported as ~ 0	Read velocity just before contact
Jittery commands make the drone sink (AerialGym motor asymmetry)	Policies descend faster than commanded	Documented, tested, diagnosed (Section 8.2)
Vertical limit unspecified (± 3 m/s)	Policies dived at 4 m/s	D16: ± 1 m/s
16 GPU-rendered environments + training exceeded 4 GB	Out-of-memory crash	D18: CPU rendering, same overall speed
Motor integration differs from AerialGym's default (Euler vs RK4)	Small dynamics difference	Measured: < 0.06 m/s effect; recorded as a deviation
Duplicate SPEC downloads (SPEC(1).md, SPEC(2).md) led to committing an old SPEC	Repository SPEC out of date	Every file now ships with a unique name and an md5 check

7. Files: full paths and purpose

Repository root on the lab desktop: ~/mtp/drone-landing-rl/. All V5 work is in ~/mtp/drone-landing-rl/v5_shin/; paths below are relative to that folder. Nothing outside it is changed except .gitignore (checkpoints *.pt/*.zip/*.pth/*.pkl under runs/ are ignored).

7.1 Specification, reports, assets

Path	Purpose
SPEC.md	Master specification and lab notebook: paper parameters with status tags, decisions D1-D18, per-block test results, and every run and evaluation with checkpoint, timestep and task.
reports/	Status reports (PDF); this document supersedes the earlier ones.
assets/lmf2.urdf	LMF2 drone: one merged link, 1.24 kg, inertia diag(0.0134, 0.0144, 0.0138) kg·m ² , 0.5 m collision cube.
assets/ground_tex/	50 procedurally generated ground textures (domain randomisation).
assets/pad_box.obj, ground_quad_s*.obj (generated)	Pad mesh with the ArUco marker on top; one ground mesh per texture scale.

7.2 Environment: envs/

Path	Purpose
envs/lmf2_params.py	Drone constants (mass, inertia, motor limits 0.1–10 N, motor time constants), controller gain ranges and sampling, timing (100 Hz physics, 10 substeps per 0.1 s action).
envs/lee_controller.py	Lee geometric velocity controller, a numpy port of AerialGym's: velocity + yaw-rate command → thrust and torques.
envs/quad.py	Drone in PyBullet: motor model in rotor-speed space (spin-up 50–80 ms, spin-down 5 ms; diagnostic modes sym_slow / sym_fast), wrench application, disturbances, state; PyBullet client set-up.
envs/platform.py	Moving platform: random-walk speed and turning rate (Table I), curriculum scaling, pad geometry (top at 1.0 m).
envs/camera.py	Pinhole camera 512 × 320, 90° horizontal FOV, pitched 60° down; grayscale; colour scaling and light direction; projection helper.
envs/pad_marker.py	ArUco texture and pad mesh generation; marker corner geometry; OpenCV detector (tests and future baseline H).
envs/ground.py	Textured ground plane with randomised texture, scale and brightness.
envs/dr.py	Domain-randomisation settings and samplers (forces, torques, initial rates, visuals); sensor-noise model.
envs/landing_sim.py	Simulation core: independent seeded random streams, spawn sampling, physics stepping, outcomes, touchdown logging (pre-contact velocity, strict success).
envs/shin_env.py	Gymnasium environment: observation dictionary (image, u, critic, target, s_rel), action scaling, which inputs each variant's actor may see.
envs/reward.py	Table III reward: lateral and vertical progress, vertical-speed penalty (literal and prose readings), undershoot, yaw rate, ±10 terminals.
envs/curriculum.py	Levels 10...80, promotion at ≥ 80% over 512-episode windows, promotion statistics.

7.3 Learning: policies/

Path	Purpose
policies/encoder.py	Image downsampling ($512 \times 320 \rightarrow 256 \times 160$) and the 5-layer CNN (2.9M parameters).
policies/shin_policy.py	Full network: CNN $\rightarrow$ LSTM $\rightarrow$ estimation layer (first 6 outputs = estimate of s_{rel}) $\rightarrow$ decision layer; critic on the true state; variant P actor; true-state injection switch for diagnostics. 5.85M parameters.
policies/ppo.py	Recurrent PPO: rollouts with stored LSTM states, advantage estimation with timeout bootstrapping, estimation loss, gradient accumulation for the 4 GB GPU.
policies/active_perception.py	Active-perception reward (Block 13).
policies/oracle.py	Scripted controller on the true state (proves solvability) and a random policy.

7.4 Scripts: scripts/

Path	Purpose
scripts/train_ppo.py	Training. Main options: --mode vision privileged, --name, --seed, --total-steps, --lambda-est, --r-active auto off, --vz-max, --vz-penalty literal prose, --motor asym sym_slow sym_fast, --renderer tiny egl, --device, --resume. Writes runs/<name>/.
scripts/evaluate.py	Pinned evaluation (seeds 9000 + i): success with 95% CI, strict and safe success, outcomes, touchdown and commanded descent speed, estimation error overall and by time out of view; --motor and --inject-true-state diagnostics.
scripts/bench_env.py	Environment throughput benchmark (renderer $\times$ resolution $\times$ number of environments).
scripts/gpu_update_check.py	Peak GPU memory and time of one PPO update.
scripts/reward_profile.py	Reward landscape for fixed descent rates, hovering and the oracle.
scripts/probe_encoder.py	Supervised probe: can the CNN find the pad from one frame?
scripts/eval_oracle.py	Oracle evaluation with domain randomisation on or off.

7.5 Tests: tests/ (175 tests, all passing)

Path	Tests	What it checks
tests/test_block1_controller.py	31	Mass/inertia, hover, step responses, heading frame, yaw clamp, motor time constants, documented jitter sink.
tests/test_block2_platform.py	10	Platform kinematics and distributions (KS tests).
tests/test_block3_camera.py	18	Camera geometry, marker detection and corner accuracy, both renderers.
tests/test_block4_termination.py	12	Every outcome, spawn distributions, success region.
tests/test_block5_oracle.py	4	Oracle solvability, random-policy reachability.
tests/test_block6_dr.py	17	Domain randomisation statistics, sensor noise, rendering of randomised visuals.
tests/test_block7_encoder.py	5	Downsampling, encoder shapes and gradients, overfit check.
tests/test_block8_env.py	12	Gymnasium compliance, observations, privilege boundary, action scaling and limits.
tests/test_block9_reward.py	19	Hand-computed reward terms, both readings, terminals.
tests/test_block10_curriculum.py	9	Promotion rule, stale episodes, state saving.
tests/test_block11_network.py	10	Dimensions, privilege boundary, recurrence, gradient routing.
tests/test_block12_ppo.py	10	Advantage estimation, timeout bootstrapping, chunked re-evaluation, accumulation, training CLI.
tests/test_block13_active.py	8	Active-perception reward hand-checked on recorded rollouts; on/off switch.
tests/test_block14_evaluate.py	3	Evaluation determinism across worker counts; vision metrics.
tests/test_diag_motor.py	5	Motor modes, unchanged random streams, jitter sink by mode, evaluation and training flags.
tests/test_diag_inject.py	2	True-state injection semantics and evaluation flag.

7.6 Runs: runs/<name>/

Each run folder holds config.json (all settings), updates.csv (one row per PPO update), episodes.csv, curriculum.csv, checkpoints (latest.pt, u<N>.pt, level_<L>.pt; not in git) and one eval_latest_c<c>_s9000_det[_suffix]/ folder per evaluation (summary.json, episodes.csv).

Run	What it is	Status / key result
P_smoke_s1	True-state actor, ± 3 m/s	91.4% at $c = 1$; dives (-3.4 m/s)
P_smoke_vz1_s1 / _s2 / _s3	True-state actor, ± 1 m/s (default), 3 seeds	81.5 / 89.9 / 83.8% at $c = 1$; 97-100% up to $c = 0.625$
P_smoke_prose_vz1_s1	Prose reading of the speed penalty	30.5%; rejected (D17)
P_smoke_vz1_symslow_s1	Trained on symmetric motors	89.7%; impact still -1.93 m/s
A_smoke_s1	Vision, 100k-step plumbing test	Works; 57 steps/s
A_s1	Vision, proposed configuration, 4M steps	Level 50; 88.8% at $c = 0.5$, 48.5% at $c = 1$
A_noEst_s1	Vision without the estimation loss, 4M steps	Stuck at level 10; 4.5% at $c = 1$
A_s2, A_s3	Vision, seeds 2 and 3, 4M steps each	Levels 50 and 30; 52.4% and 34.7% at $c = 1$
bench_20261001_211130	Throughput benchmark	CPU vs GPU rendering speeds (D18)

8. Results

8.1 Environment validated before any learning

Check	Result
Controller	Holds hover exactly; tracks velocity steps; 10 m/s chase reached in 1 s.
Platform, camera	Motion statistics match the paper's tables; marker corners within 0.7–1.6 px of geometry.
Scripted controller (true state)	99% success at full difficulty , touchdown about -0.6 m/s: the task is solvable and gentle landing is possible.
Visibility	Pad out of view in $\sim 50\%$ of frames of a good landing: memory and estimation are essential.

8.2 Touchdown speed and the motor finding

Run (variant P, 2M steps)	Success at $c = 1$	Impact median	Safe ≤ 1 m/s
P_smoke_s1 (± 3 m/s limit)	91.4%	-3.36 m/s	0.4%
P_smoke_vz1_s1 (± 1 m/s, D16)	81.5%	-2.30 m/s	1.2%
P_smoke_prose_vz1_s1 (prose penalty)	30.5%	-1.66 m/s	2.5%
P_smoke_vz1_symslow_s1 (trained on symmetric motors)	89.7%	-1.93 m/s	4.2%

- **Why policies hit faster than they can command:** AerialGym's LMF2 motors spin down $10\text{--}16\times$ faster than they spin up. Jittery commands therefore lose thrust and the drone sinks (measured 0.7 m/s with no descent command). Evaluating the same checkpoint with symmetric motors cut touchdown from -2.30 to -0.78 m/s, and the policy then failed to come down in 136 episodes: it uses the sink as its way down.
- **But removing it in training did not give gentle landings** (-1.93 m/s): the policy found another way down. The root cause is the task definition: the reward pays for descent, the $+10$ on touchdown replaces any speed penalty, and success has no speed limit.
- **Decision:** keep the paper's motors and reward for the reproduction; report safe success next to the paper's metric. A "safe landing" variant is a separate experiment after the reproduction.

8.3 Vision run A_s1 (4M steps, ~ 20 h)

Configuration A: ArUco + CNN + LSTM + estimation loss + active perception, paper's motors and reward, ± 1 m/s. It reached curriculum level 50 ($c = 0.625$) after 4M steps; variant P reached level 80 in about 0.5M.

Pinned evaluation, latest.pt at 4.0M steps, 1,000 episodes, seed 9000. P = P_smoke_vz1_s1/latest.pt on the same episodes.

c	P (true state)	A (vision)	A + true state injected	Gap closed by injection	A estimate RMSE (pos / vel)
0.125	100.0%	93.4%	–	–	0.31 m / 0.86 m/s
0.5	99.0%	88.8%	89.1%	~ 0 of 10 pts	0.84 m / 0.93 m/s
0.625 (A's last level)	97.5%	82.2%	86.4%	~ 4 of 15 pts	1.37 m / 1.12 m/s
1.0 (A never trained)	81.5%	48.5%	54.8%	~ 6 of 33 pts	4.36 m / 2.58 m/s

- At $c = 0.5$, A's estimation error (0.84 m / 0.93 m/s) is close to the paper's for this configuration (0.95 m / 1.06 m/s).
- A's position error grows with time out of view: about 0.2 m when visible, 1–3 m after 16–60 blind steps, and 6.7 m at $c = 1$.

- A touches down at -3.5 to -4.1 m/s while commanding about -0.5 m/s; with symmetric fast motors this drops to -1.6 m/s, but most episodes then drift away. A relies on the motor sink even more than P.
- Safe landing (≤ 1 m/s) is about 0% for A at every difficulty.

8.4 Three P seeds and the A-noEst ablation (4 Oct)

Same pinned 1,000 episodes. A-noEst = A with the estimation loss switched off (so also no active perception), 4M steps, ~ 20 h; it never left level 10.

c	P s1 / s2 / s3	P mean	A_s1	A_noEst_s1
0.125	100.0 / 100.0 / 99.7%	99.9%	93.4%	25.2%
0.5	99.0 / 99.2 / 99.3%	99.2%	88.8%	7.9%
0.625	97.5 / 97.6 / 97.1%	97.4%	82.2%	7.1%
1.0	81.5 / 89.9 / 83.8%	85.1%	48.5%	4.5%

- **The control ceiling is stable:** P's three seeds agree within 1 point up to $c = 0.625$; at full difficulty they range 81.5–89.9%, so the full task is harder for control too.
- **The estimation loss is essential for learning:** A-noEst mostly crashes into the ground or flips (tilt), even at the easiest level. The paper reports the same direction (73.99% vs 91%), but the effect here is far larger. A plausible reason (untested): on the ArUco base the CNN has no pretrained keypoint features, so state supervision is the main thing shaping what the network learns to see.

8.5 Three seeds of vision configuration A (6 Oct)

Same pinned 1,000 episodes; success rates in %. Gap = P mean – A mean. Injection = A's decision layer given the true relative state instead of its own estimate.

c	P s1 / s2 / s3	A s1 / s2 / s3	A + inject s1 / s2 / s3	P mean	A mean	A+inj mean	Gap	Recovered
0.125	100 / 100 / 99.7	93.4 / 93.8 / 90.1	–	99.9	92.4	–	7.5	–
0.5	99.0 / 99.2 / 99.3	88.8 / 91.1 / 67.9	89.1 / 92.4 / 73.0	99.2	82.6	84.8	16.6	13%
0.625	97.5 / 97.6 / 97.1	82.2 / 83.8 / 56.9	86.4 / 87.2 / 58.2	97.4	74.3	77.3	23.1	13%
1.0	81.5 / 89.9 / 83.8	48.5 / 52.4 / 34.7	54.8 / 59.8 / 36.5	85.1	45.2	50.4	39.9	13%

- **The gap grows with platform motion** ($7.5 \rightarrow 40$ points), and vision's curriculum progress varies strongly by seed (levels 50 / 50 / 30), while P's does not (level 80×3).
- **Injection recovers $\sim 13\%$ of the gap at every difficulty** (3-seed mean; per seed 3–27%). The A_s1 finding holds.
- **The weakest seed (A_s3) gains least from injection** (≤ 5 points). A policy limited by its estimate would gain most, so this argues against an estimation-error explanation for the residual gap.
- **Confound still open:** P finished the curriculum; A did not. Next check: P checkpoints saved when P reached the same level as A (evaluation only).

8.6 What this says about state vs control

A two-part answer (P and A: 3 seeds; A-noEst: 1 seed). (1) **During learning, state is the bottleneck:** without explicit supervision of the relative state, the vision policy does not learn to land at all (A-noEst), while with it, it reaches 74–92% up to $c = 0.625$. (2) **After learning, the remaining gap is not in the explicit estimate:** handing A's controller the true state recovers only ~13% of its gap to P (never more than 27% in any seed). The estimate does matter more at full difficulty, where the pad is out of view for long stretches (injection halves drift failures). Vision also progressed through the curriculum about 8× more slowly.

Limits: (1) A's controller also reads 250 other internal numbers from vision, so injection corrects only the explicit estimate, not all state information. (2) A-noEst has one seed (P and A have three). (3) P was trained to full difficulty, A only to $c = 0.625$. (4) All on the ArUco base, not the paper's keypoint pad.

9. What we are doing now

Just finished (6 Oct): A_s2 and A_s3 with pinned evaluations and the injection test (commit 8d94334).

Proposed next steps (each needs approval):

Step	What	Cost	Why
1	Curriculum-matched comparison: evaluate P's level_60.pt (trained through level 50) and level_40.pt (through level 30) on the same episodes	minutes, evaluation only	Removes the confound that P finished the curriculum and A did not
2	A longer A run (~8M steps) to reach full difficulty	~40 h	Compare with the paper's 91% at $c = 1$
3	A-noEst seeds 2–3	~40 h	Rule D14; the effect is large, so lower priority
4	Stronger state test: retrain only A's decision layer on the true state, everything else frozen	~1 day work + run	Addresses the 250-latent caveat of injection
5	Baseline H (ArUco + EKF + RL)	build + runs	Paper's model-based comparison

10. Status of every planned item

Item	Status
P seeds 1–3	Done. Stable ceiling (Section 8.4).
A-noEst_s1 (4M steps)	Done. Never left level 10; 4.5% at $c = 1$ (Section 8.4).
A at full difficulty	Not reached. A longer run (~8M steps, ~40 h) is needed to compare with the paper's 91% at $c = 1$.
A-noAP, baseline H (EKF + RL)	Not started.
Seeds for vision runs	Done for A (3 seeds). A-noEst has 1 seed.
Variant K (PACMAN keypoint pad)	Blocked: weights not public; requested from the authors; fallback is training our own detector.
Safe-landing variant	After the reproduction, as a single change (speed condition in success or impact-speed penalty).

Questions where a second opinion would help

- Is true-state injection a fair test of state vs control, given the 250 other latent numbers? A stronger test would be to retrain A's decision layer on the true state while freezing the rest. Is it worth the extra run?
- Should the thesis report the paper's success metric, safe success, or both as the main number?

- Given about 20 h per vision run on this desktop, which matters more next: A to full difficulty, more seeds, or the ablations (A-noEst, H)?
- How do the paper's authors get $\lesssim 1$ m/s descents with this reward and AerialGym's motors? Their code is not public.

11. How to reproduce and monitor

```
cd ~/mtp/drone-landing-rl && source drone-landing-rl-venv/bin/activate
python -m pytest v5_shin/tests -q                                # 175 tests
python -m v5_shin.scripts.train_ppo --mode privileged --name P_smoke_vz1_s1 --seed 1 \
    --total-steps 2000000 --renderer tiny                        # variant P, ~25 min
python -m v5_shin.scripts.train_ppo --mode vision --name A_s1 --seed 1 \
    --total-steps 4000000 --renderer tiny --device cuda         # variant A, ~20 h
python -m v5_shin.scripts.evaluate --run A_s1 --ckpt latest.pt --episodes 1000 \
    --c 0.5 0.625 1.0 --renderer tiny [--inject-true-state] [--motor sym_fast]
```

Full detail for every block, decision and run is in `v5_shin/SPEC.md`. All results are committed in the repository.